

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL 08 – Asynchronous JavaScript (Fetch API,
JSON, REST API Sederhana)**

Nama Mahasiswa : Saprina Melita Sari
NIM : 23051430012
Kelas : A
Tanggal Praktikum : 17 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 8 (Pertemuan 8)
Topik Utama	Form Validation & Local Storage
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome	Versi terbaru	Testing & Debugging
3	Live Server	Versi 5.7.9	Extentions VS Code untuk menjalankan html

4	Prettier	Versi 12.3.0	Extenions VS Code untuk merapikan indentansi code
5	Bootstrap	Versi 5.3.8	Framework front-end open-source untuk mempercepat pengembangan website

D. DASAR TEORI

D.1 Konsep Utama

Modul ini secara khusus membahas penerapan Asynchronous JavaScript dan penggunaan Fetch API sebagai dasar arsitektur komunikasi data antara sisi klien (client-side) dengan server eksternal (server-side). Secara konsep, eksekusi kode asinkron memungkinkan aplikasi web mengirim serta memproses permintaan data di latar belakang tanpa menghambat atau menghentikan (blocking) interaksi pengguna pada antarmuka utama. Pada praktikum ini, konsep tersebut diterapkan ketika program mengambil daftar laporan dari REST API publik secara terstruktur, yang kemudian didukung dengan adanya indikator loading sebagai bentuk umpan balik visual kepada pengguna selama proses pertukaran data melalui jaringan berlangsung.

Selain itu, modul ini juga menekankan pengelolaan standar pertukaran data modern menggunakan JavaScript Object Notation (JSON) beserta proses transformasi datanya. Data mentah yang diterima dari server perlu melalui proses parsing JSON agar dapat diubah menjadi struktur objek yang mudah dibaca dan diolah oleh sistem. Setelah proses ekstraksi selesai, program memperlihatkan teknik mapping dan lokalisasi dengan membatasi jumlah data array yang ditampilkan serta menggantinya menggunakan teks simulasi berbahasa Indonesia. Hal tersebut menunjukkan bahwa data dari pihak ketiga dapat dimodifikasi dan disesuaikan agar lebih relevan dengan kebutuhan operasional bisnis yang sedang dikembangkan.

Pada bagian akhir, modul ini mengintegrasikan manipulasi Document Object Model (DOM) secara dinamis yang dikombinasikan dengan prinsip fault-tolerance melalui mekanisme error handling. Data yang telah distandardisasi kemudian ditampilkan secara otomatis menjadi elemen visual berbentuk kartu informasi, di mana setiap kartu memiliki identitas unik (ID) agar interaksi pada tiap elemen dapat berjalan dengan tepat. Sebagai bentuk pengamanan sistem, implementasi pengecekan status respons serta penggunaan blok catch memastikan bahwa ketika terjadi gangguan konektivitas, aplikasi tidak mengalami crash, melainkan tetap berjalan stabil dan menampilkan pesan kesalahan yang informatif kepada pengguna.

D.2 Keterkaitan dengan Teknik Industri

Praktikum pada modul ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, terutama dalam pengembangan Sistem Informasi Manajemen (SIM) dan sistem pemantauan produksi berbasis real-time. Dalam dunia manufaktur yang terus bergerak dinamis, informasi terkait kondisi shop floor harus dapat diakses secara cepat tanpa menghambat aktivitas operasional lainnya. Implementasi Asynchronous JavaScript dan Fetch API pada program ini menggambarkan konsep dasbor pemantauan industri atau Andon system yang mampu mengambil data, seperti kondisi kerusakan mesin maupun anomali sensor, dari server pusat atau sistem Enterprise Resource Planning (ERP) yang berjalan di latar belakang. Kemampuan sistem untuk memperbarui tampilan secara asinkron tanpa menyebabkan lag atau pembekuan layar menjadi faktor penting agar operator maupun manajer produksi dapat memantau performa proses secara terus-menerus dan responsif.

Selain itu, hubungan dengan keilmuan Teknik Industri juga tampak pada proses standardisasi dan lokalisasi data operasional. Pada era Industrial Internet of Things (IIoT), berbagai mesin, sensor, dan subsistem perangkat lunak saling bertukar data mentah dalam jumlah besar. Tahapan pengambilan data dari REST API, proses parsing JSON, hingga penyesuaian struktur informasi menjadi laporan insiden yang terstandardisasi dalam program ini mencerminkan prinsip interoperabilitas pada sistem terintegrasi. Mekanisme tersebut menggambarkan bagaimana data yang berasal dari perangkat eksternal atau vendor dapat diterjemahkan dan disesuaikan dengan kebutuhan operasional perusahaan. Standarisasi data seperti ini menjadi kompetensi penting bagi insinyur industri dalam membangun sistem informasi yang valid guna mendukung analisis performa mesin dan pengambilan keputusan manajerial.

Di sisi lain, interaktivitas antarmuka yang diwujudkan melalui manipulasi *Document Object Model* (DOM) pada program ini merepresentasikan alur kerja *Computerized Maintenance Management System* (CMMS) yang efektif dan minim kesalahan. Saat tombol “Tindak Lanjut” pada kartu insiden dipilih, sistem secara otomatis memberikan respons visual sekaligus menonaktifkan interaksi lanjutan pada elemen tersebut. Mekanisme ini merupakan penerapan konsep Poka-Yoke atau sistem pencegah kesalahan, yang bertujuan menghindari duplikasi tiket perbaikan maupun benturan penugasan pada tim *maintenance*. Dengan

dukungan *error handling* untuk menjaga kestabilan aplikasi ketika terjadi gangguan jaringan, rancangan program ini mendukung prinsip Teknik Industri dalam menciptakan sistem kerja yang terintegrasi, efisien, mengurangi *waste of waiting*, serta memiliki tingkat keandalan operasional yang tinggi.

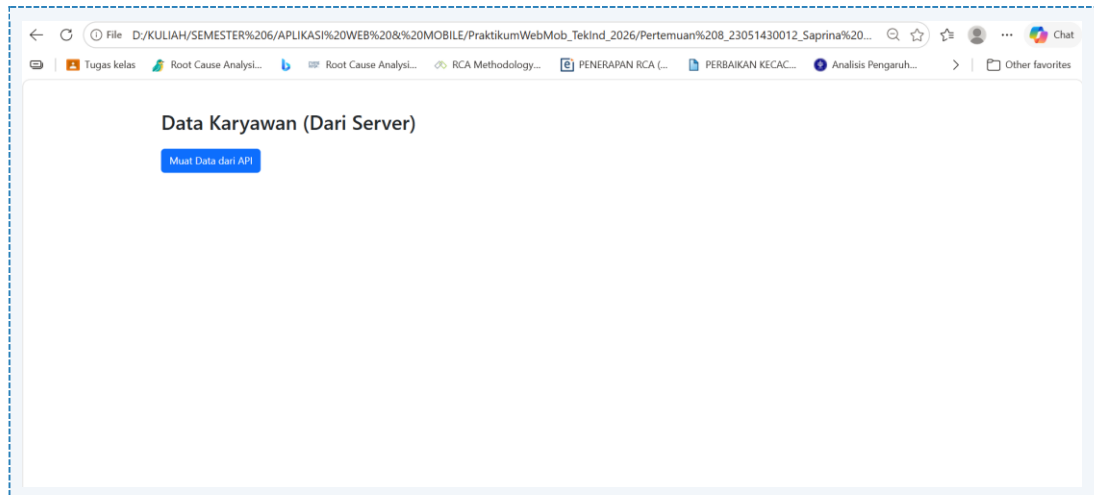
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat HTML untuk Menampilkan Data

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootst
trap.min.css" rel="stylesheet">
</head>
<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data
dari API</button>
    <div id="loading" class="spinner-border text-primary d-
none" role="status"></div>
  </div>
  <div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
  </div>
  <script src="api_script.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Membuat html untuk menampilkan data

E.2 Langkah 2 – Fetch Data dengan .then() (Promise)

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error (misal putus internet)
```

```
        container.innerHTML = `

### Screenshot Hasil:

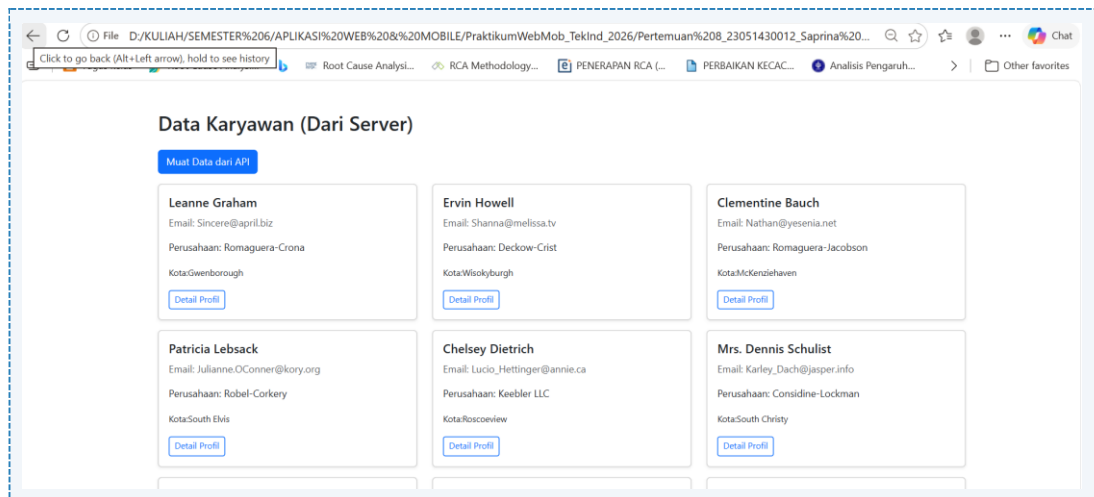


Saprina Melita Sari – 23051430012 | Modul 08



Halaman 7 dari 21


```



Gambar E.2 – hasil fetch then()(Promise)

E.3 Langkah 3 – Fetch dengan Async/Await (Modern Style)

```
// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);

    const response = await
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

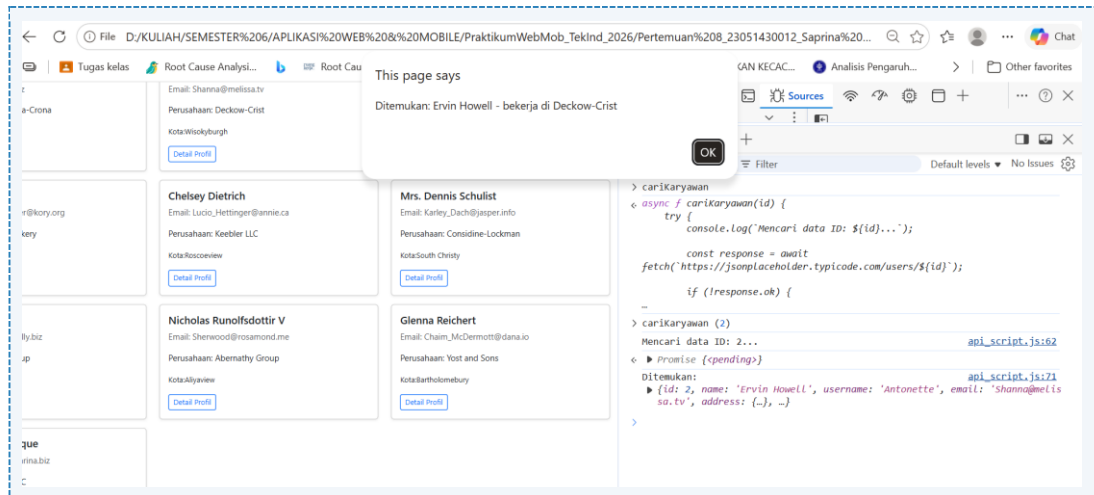
    if (!response.ok) {
      throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
    console.log("Ditemukan:", data);
    alert(`Ditemukan: ${data.name} - bekerja di
    ${data.company.name}`);

  } catch (error) {
    console.error(error);
    alert(error.message);
  }
}

// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.
```

Screenshot Hasil:



Gambar E.3 – Fetch dengan Async/Await

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API.

Jawaban / Kode Solusi:

```

// --- 3. FUNGSI TAMBAH DATA (POST) ---
formTambah.addEventListener('submit', function (e) {
  e.preventDefault();

  const dataBaru = {
    name: document.getElementById('inputNama').value,
    email: document.getElementById('inputEmail').value,
    company: { name:
document.getElementById('inputPerusahaan').value },
    address: { city:
document.getElementById('inputKota').value }
  };

  fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(dataBaru)
  })
    .then(function (response) {
  
```

```
        if (!response.ok) throw new Error('Gagal menambah data');
        return response.json();
    })
    .then(function (result) {
        // Tutup Modal
        const modalInstance =
bootstrap.Modal.getInstance(document.getElementById('modalTambah'
));
        modalInstance.hide();
        formTambah.reset();

        // Tampilkan card baru di paling atas
        renderData([result], true);

        // Masukkan data baru ke variabel daftarKaryawan agar
sinkron saat difilter nanti
        daftarKaryawan.unshift(result);

        alert("Karyawan baru berhasil ditambahkan!");
    })
    .catch(function (error) {
        alert("Error: " + error.message);
    });
});
```

The screenshot displays a web application interface with a modal form titled "Input Karyawan Baru". The modal is overlaid on a background that shows a "Data Karyawan" section with buttons for "MUAT DATA DARI API" and "FILTER". The modal form contains the following fields:

- NAMA LENGKAP:** A text input field with the placeholder text "Misal: Rajeshwari".
- EMAIL:** A text input field with the placeholder text "rajeshwari@email.com".
- NAMA PERUSAHAAN:** A text input field with the placeholder text "PT Suzuki".
- KOTA:** A text input field with the placeholder text "Kota Baru".

At the bottom of the modal, there are two buttons: "BATAL" (Cancel) and "SIMPAN DATA" (Save Data).

Gambar F.1 – Hasil pembuatan form dan button tambah karyawan

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

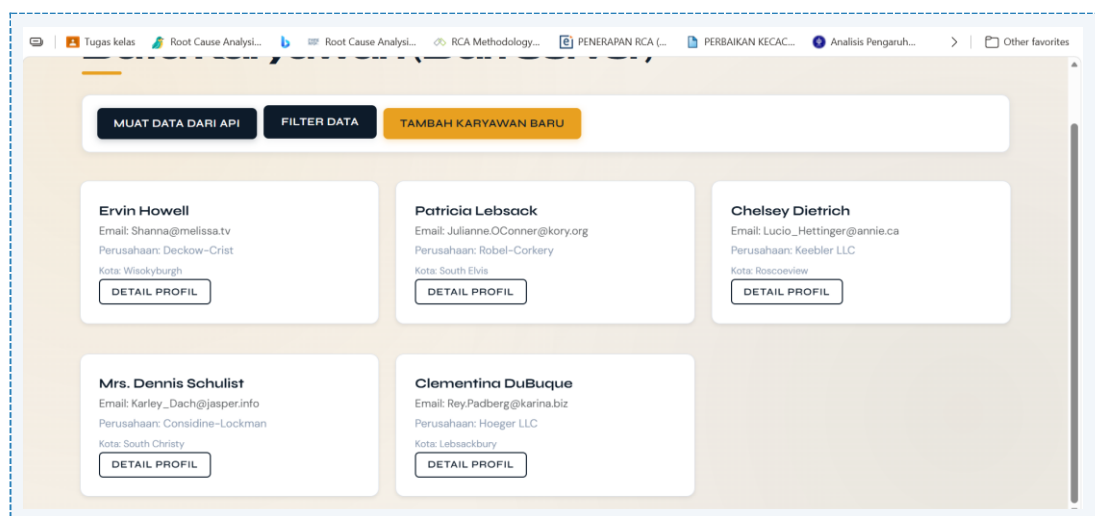
Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

```
// --- 2. FUNGSI FILTER KOTA DENGAN HURUF 'S' ---
btnFilter.addEventListener('click', function () {
  // Validasi jika data belum dimuat
  if (daftarKaryawan.length === 0) {
    alert("Harap klik 'Muat Semua Data' terlebih dahulu!");
    return;
  }

  // Filter dari variabel daftarKaryawan
  const hasilFilter = daftarKaryawan.filter(function (item) {
    return item.address.city.toLowerCase().includes('s');
  });

  // Tampilkan hasil filter ke layar
  container.innerHTML = '';
  renderData(hasilFilter);
});
```



Gambar F.2 – Hasil pembuatan button filter data

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

Anggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan insiden".

- Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
- Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance"

Penjelasan pendekatan/solusi yang digunakan:

Solusi yang digunakan pada program ini menerapkan konsep Asynchronous JavaScript dengan memanfaatkan fetch API, async/await, dan Promise.all() untuk mengambil data insiden serta data karyawan dari server secara paralel tanpa menghambat interaksi pengguna pada halaman web. Selama proses pengambilan data berlangsung, sistem menampilkan indikator loading sebagai umpan balik visual agar pengguna mengetahui bahwa aplikasi sedang memproses permintaan jaringan. Data yang diterima dalam format JSON kemudian diproses dan ditampilkan secara dinamis menggunakan manipulasi DOM melalui fungsi render sehingga halaman dapat diperbarui tanpa perlu melakukan reload. Program juga menerapkan pengolahan array menggunakan metode filter() dan forEach(), di mana data karyawan disimpan terlebih dahulu pada variabel semuaKaryawan, kemudian difilter berdasarkan nama kota menggunakan includes() sebelum dirender kembali ke layar. Selain itu, aplikasi menyediakan fitur penambahan karyawan baru melalui form modal Bootstrap dengan menambahkan object baru ke dalam array menggunakan push(), sehingga data dapat langsung tampil secara real-time. Fitur detail profil karyawan juga diimplementasikan menggunakan modal popup untuk menampilkan informasi lengkap berdasarkan data yang dipilih pengguna. Sebagai bentuk ketahanan sistem, program menggunakan mekanisme try...catch untuk menangani error saat proses pengambilan data, sehingga aplikasi tetap stabil dan mampu memberikan pesan kesalahan yang jelas apabila terjadi gangguan koneksi atau kegagalan server.

Kode Utama:

```
const API_POSTS = "https://jsonplaceholder.typicode.com/posts";
const API_USERS = "https://jsonplaceholder.typicode.com/users";

let semuaKaryawan = [];

// Ambil elemen
const btnLoad = document.getElementById("btnLoad");
const btnFilter = document.getElementById("btnFilter");
const loading = document.getElementById("loading");
const cInsiden = document.getElementById("containerInsiden");
const cKaryawan = document.getElementById("containerKaryawan");
const statusBadge = document.getElementById("statusBadge");
```

```

const modalDetailBS = new
bootstrap.Modal(document.getElementById("modalDetail"));

// ===== UTILITY =====
const setLoading = (v) => {
  loading.classList.toggle("d-none", !v);
  btnLoad.disabled = v;
};

const inisial = (nama) =>
  nama.split(" ").slice(0, 2).map(w => w[0].toUpperCase()).join("");

const getStatus = (id) => {
  const list = [
    { label: "Kritis", cls: "badge-kritis" },
    { label: "Sedang", cls: "badge-sedang" },
    { label: "Rendah", cls: "badge-rendah" },
  ];
  return list[id % list.length];
};

// ===== INSIDEN (BAHASA INDONESIA) =====
function renderInsiden(posts) {
  cInsiden.innerHTML = "";

  const data = posts.slice(0, 10);
  document.getElementById("jumlahInsiden").textContent =
    `${data.length} insiden`;

  const daftarMasalah = [
    "Kerusakan mesin produksi",
    "Keterlambatan proses produksi",
    "Kesalahan input bahan baku",
    "Gangguan sistem operasional",
    "Penurunan kualitas produk"
  ];

  data.forEach((post, i) => {
    const status = getStatus(post.id);

    const judul = daftarMasalah[post.id % daftarMasalah.length];
    const deskripsi = `Insiden berupa ${judul.toLowerCase()} sedang dalam
proses penanganan oleh tim produksi. (ID: ${post.id})`;

    cInsiden.innerHTML += `
      <div class="col-md-6 col-lg-4">

```

```

        <div class="card-insiden">
            <div class="insiden-meta">
                <span>TKT-${String(post.id).padStart(4,"0")}</span>
                <span class="${status.cls}">${status.label}</span>
            </div>

            <p class="insiden-title"><b>${judul}</b></p>
            <p class="insiden-desc">${deskripsi}</p>

            <div class="insiden-footer">
                <span>Pelapor: User #${post.userId}</span>
                <button class="btn-tindak" data-id="${post.id}">
                    Tindak Lanjut
                </button>
            </div>
        </div>
    </div>
    `;
    });
}

// Klik tindak lanjut
cInsiden.addEventListener("click", (e) => {
    if (e.target.classList.contains("btn-tindak")) {
        alert(`Insiden ${e.target.dataset.id} sedang diproses oleh tim
maintenance`);
    }
});

// ===== KARYAWAN =====
function renderKaryawan(data) {
    cKaryawan.innerHTML = "";

    document.getElementById("jumlahKaryawan").textContent =
        `${data.length} karyawan`;

    if (data.length === 0) {
        cKaryawan.innerHTML = `
            <div class="col-12 text-center">
                <p>Tidak ada data karyawan</p>
            </div>`;
        return;
    }

    data.forEach((user) => {
        cKaryawan.innerHTML += `

```

```

        <div class="col-md-6 col-lg-4">
            <div class="card-karyawan">
                <div class="avatar">${inisial(user.name)}</div>
                <p><b>${user.name}</b></p>
                <p>${user.email}</p>
                <p>${user.phone}</p>
                <p>${user.address.city}</p>

                <button class="btn-detail"
                    data-user="${encodeURIComponent(JSON.stringify(user))}">
                    Lihat Detail
                </button>
            </div>
        </div>
    `;
});
}

// Detail modal
cKaryawan.addEventListener("click", (e) => {
    if (!e.target.classList.contains("btn-detail")) return;

    const user = JSON.parse(
        decodeURIComponent(e.target.dataset.user)
    );

    document.getElementById("detailNama").textContent = user.name;
    document.getElementById("detailUsername").textContent = user.username;
    document.getElementById("detailEmail").textContent = user.email;
    document.getElementById("detailTelepon").textContent = user.phone;
    document.getElementById("detailWebsite").textContent = user.website;
    document.getElementById("detailPerusahaan").textContent = user.company?.name
    ?? "-";
    document.getElementById("detailAlamat").textContent =
        `${user.address.street}, ${user.address.suite}, ${user.address.city}
        ${user.address.zipcode}`;

    modalDetailBS.show();
});

// ===== FETCH DATA =====
async function muatData() {
    setLoading(true);

    try {
        const [resPosts, resUsers] = await Promise.all([

```

```
        fetch(API_POSTS),
        fetch(API_USERS)
    ]);

    if (!resPosts.ok) throw new Error("Gagal mengambil data insiden");
    if (!resUsers.ok) throw new Error("Gagal mengambil data karyawan");

    const posts = await resPosts.json();
    const users = await resUsers.json();

    semuaKaryawan = users;

    renderInsiden(posts);
    renderKaryawan(users);

    statusBadge.className = "status-badge status-online";
    statusBadge.innerHTML = `<span class="status-dot"></span> Online`;
  } catch (err) {
    alert(err.message);
  }

  setLoading(false);
}

// ===== FILTER =====
btnFilter.addEventListener("click", () => {
  if (semuaKaryawan.length === 0) {
    alert("Muat data terlebih dahulu");
    return;
  }

  const keyword = prompt("Masukkan nama kota:").toLowerCase();
  if (!keyword) return;

  const hasil = semuaKaryawan.filter(u =>
    u.address.city.toLowerCase().includes(keyword)
  );

  renderKaryawan(hasil);
});

// ===== TAMBAH KARYAWAN =====
document.getElementById("formTambahKaryawan").addEventListener("submit", (e)
=> {
  e.preventDefault();
```

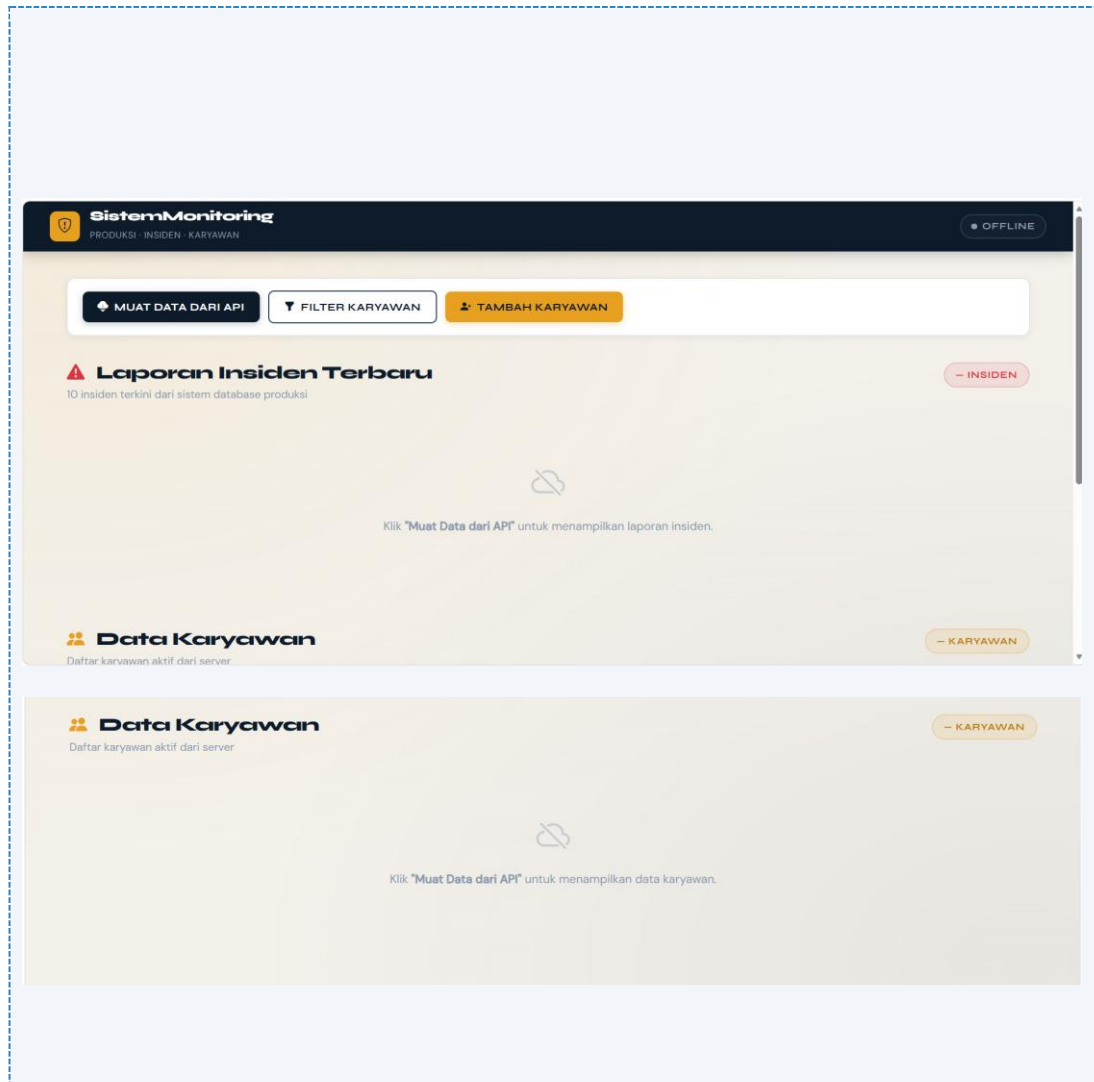


```
const karyawanBaru = {
  id: semuaKaryawan.length + 1,
  name: document.getElementById("inputNama").value,
  email: document.getElementById("inputEmail").value,
  phone: "-",
  username:
document.getElementById("inputNama").value.toLowerCase().replace(/\s+/g, "."),
  website: "-",
  company: { name: document.getElementById("inputPerusahaan").value },
  address: {
    street: "-",
    suite: "-",
    city: document.getElementById("inputKota").value,
    zipcode: "-"
  }
};

semuaKaryawan.push(karyawanBaru);
renderKaryawan(semuaKaryawan);

bootstrap.Modal.getInstance(document.getElementById("modalTambah"))?.hide();
e.target.reset();
});

// ===== LOAD =====
btnLoad.addEventListener("click", muatData);
```



Gambar F.3 – Hasil pembuatan mini proyek integrasi data produksi

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Kesesuaian antara hasil program dengan teori menunjukkan bahwa implementasi yang dibuat telah selaras dengan konsep dasar Asynchronous JavaScript dalam komunikasi data eksternal, di mana proses asinkron memungkinkan eksekusi kode berjalan di latar belakang sehingga antarmuka aplikasi tetap responsif dan tidak mengalami blocking saat menunggu respons dari server. Pada program Laporan Insiden ini, penerapan tersebut terlihat melalui penggunaan Fetch API untuk mengambil data dari REST API publik, yang didukung dengan indikator loading sebagai umpan balik visual selama proses pengambilan data berlangsung. Selain itu, kesesuaian teori juga tampak pada pengelolaan Promise, parsing data JSON, dan mekanisme error handling, karena data respons dari server terlebih dahulu diubah ke dalam format objek JSON agar dapat dimanipulasi dan ditampilkan pada antarmuka aplikasi. Implementasi blok penanganan kesalahan juga membuktikan penerapan prinsip ketahanan sistem, di mana aplikasi tidak mengalami crash ketika terjadi kegagalan request, melainkan mampu menampilkan pesan

peringatan yang informatif kepada pengguna. Di sisi lain, integrasi antara arsitektur asinkronus dan manipulasi DOM berhasil merealisasikan konsep aplikasi front-end modern yang berfungsi sebagai dasbor interaktif untuk menghubungkan pengguna dengan sumber data terpusat. Transformasi data menjadi simulasi laporan insiden industri menunjukkan implementasi yang relevan dengan kebutuhan sistem operasional modern, khususnya dalam integrasi dengan database ERP maupun jaringan sensor IoT guna mendukung proses monitoring dan maintenance yang lebih cepat, efisien, serta terpantau secara real-time.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Saat melakukan request menggunakan <code>fetch()</code> , data yang dikembalikan masih berbentuk objek Promise dengan status pending, sehingga data belum dapat dirender ke dalam DOM dan tampilan aplikasi terlihat kosong.	Menggunakan metode <code>.then()</code> maupun <code>async/await</code> agar proses render elemen hanya dijalankan setelah Promise berhasil diselesaikan (resolved) dan data dari server telah diterima sepenuhnya.
Setelah menambahkan data baru, tampilan tidak langsung diperbarui sehingga harus memuat ulang halaman.	Menambahkan data baru ke dalam array menggunakan <code>push()</code> lalu merender ulang data secara otomatis agar perubahan langsung terlihat
Saat tombol “Tindak Lanjut” ditekan, aplikasi gagal mengenali elemen target sehingga perubahan tampilan tidak terjadi pada elemen yang sesuai.	Memberikan ID unik secara dinamis pada setiap elemen HTML, misalnya <code>id="card-\${laporan.id}"</code> , sehingga event listener dapat memanipulasi elemen tertentu menggunakan <code>document.getElementById()</code> .

G.3 Kaitan dengan Penerapan di Industri

Kegiatan praktikum pada modul ini memiliki keterkaitan yang kuat dengan sistem informasi industri, khususnya dalam komunikasi data terpusat, pemantauan operasional real-time, dan manajemen insiden berbasis digital. Implementasi Asynchronous JavaScript menggunakan Fetch API memungkinkan proses pengambilan data dari server berjalan di latar belakang tanpa menyebabkan antarmuka operator mengalami blocking, sehingga aktivitas monitoring tetap responsif dan lancar. Integrasi REST API, pengolahan data JSON, serta proses standarisasi informasi mencerminkan konsep interoperabilitas pada sistem Industrial Internet of Things (IIoT) dan Enterprise Resource Planning (ERP), di mana data dari server eksternal dapat diproses dan disesuaikan menjadi laporan operasional yang lebih relevan.

Selain itu, penerapan error handling meningkatkan keandalan sistem dengan mendeteksi kegagalan koneksi dan memberikan peringatan kepada pengguna, sedangkan manipulasi DOM secara dinamis mendukung alur kerja incident management melalui pembaruan status real-time, perubahan visual elemen, dan penonaktifan tombol untuk mencegah duplikasi penanganan. Secara keseluruhan, rancangan program ini merepresentasikan kebutuhan industri modern dalam membangun sistem pelaporan dan monitoring yang terintegrasi, responsif, stabil, serta terkoordinasi dengan baik.

H. KESIMPULAN

1	Fungsi fetch() digunakan untuk mengambil data dari REST API secara asinkronus sehingga halaman tetap responsif tanpa mengalami blocking.
2	Pemberian ID unik pada elemen memungkinkan perubahan tampilan dan status tombol dilakukan secara tepat pada elemen tertentu tanpa memengaruhi elemen lain.
3	Metode .json() berfungsi mengubah respons server menjadi objek JSON agar data dapat diproses oleh program.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://www.javascripttutorial.net/
[3]	https://www.javascripttutorial.net/javascript-dom/
[4]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[5]	https://www.w3schools.com/js/default.asp

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
	
Saprina Melita Sari NIM:23051430012	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____